CHAPTER 24

Developing a Career Path

Becoming an architect takes time and effort, but based on the many reasons we've outlined throughout this book, managing a career path after becoming an architect is equally tricky. While we can't chart a specific career path for you, we can point you to some practices that we have seen work well.

An architect must continue to learn throughout their career. The technology world changes at a dizzying pace. One of Neal's former coworkers was a world-renowned expert in Clipper. He lamented that he couldn't take the enormous body of (now useless) Clipper knowledge and replace it with something else. He also speculated (and this is still an open question): has any group in history learned and thrown away so much detailed knowledge within their lifetimes as software developers?

Each architect should keep an eye out for relevant resources, both technology and business, and add them to their personal stockpile. Unfortunately, resources come and go all too quickly, which is why we don't list any in this book. Talking to colleagues or experts about what resources they use to keep current is one good way of seeking out the latest newsfeeds, websites, and groups that are active in a particular area of interest. Architects should also build into their day some time to maintain breadth utilizing those resources.

The 20-Minute Rule

As illustrated in Figure 2-6, technology breadth is more important to architects than depth. However, maintaining breadth takes time and effort, something architects should build into their day. But how in the world does anyone have the time to actually go to various websites to read articles, watch presentations, and listen to podcasts? The answer is…not many do. Developers and architects alike struggle with the balance of working a regular job, spending time with the family, being available for

our children, carving out personal time for interests and hobbies, and trying to develop careers, while at the same time trying to keep up with the latest trends and buzzwords.

One technique we use to maintain this balance is something we call the *20-minute rule*. The idea of this technique, as illustrated in Figure 24-1, is to devote *at least* 20 minutes a day to your career as an architect by learning something new or diving deeper into a specific topic. Figure 24-1 illustrates examples of some of the types of resources to spend 20 minutes a day on, such as InfoQ, DZone Refcardz, and the ThoughtWorks Technology Radar. Spend that minimum of 20 minutes to Google some unfamiliar buzzwords ("the things you don't know you don't know" from Chapter 2) to learn a little about them, promoting that knowledge into the "things you know you don't know." Or maybe spend the 20 minutes going deeper into a particular topic to gain a little more knowledge about it. The point of this technique is to be able to carve out some time for developing a career as an architect and continuously gaining technical breadth.

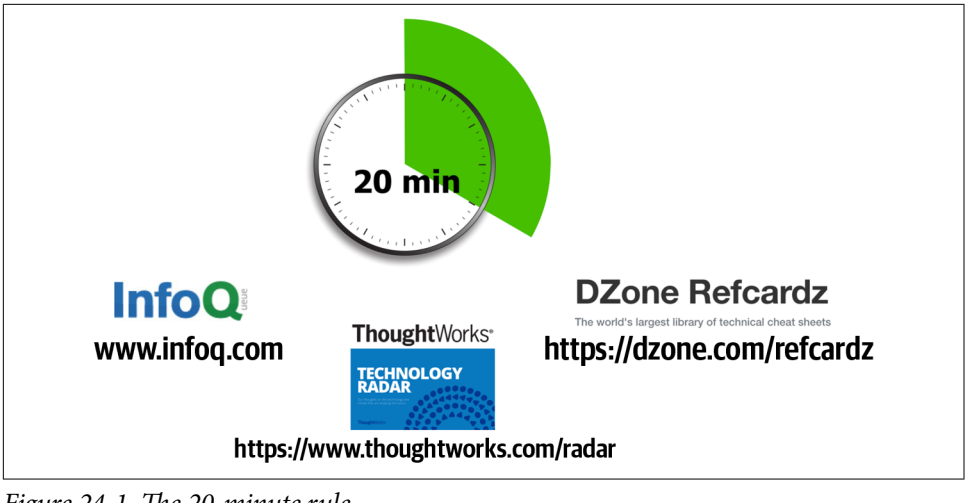

Figure 24-1. The 20-minute rule

Many architects embrace this concept and plan to spend 20 minutes at lunch or in the evening after work to do this. What we have experienced is that this rarely works. Lunchtime gets shorter and shorter, becoming more of a catch-up time at work rather than a time to take a break and eat. Evenings are even worse—situations change, plans get made, family time becomes more important, and the 20-minute rule never happens.

We strongly recommend leveraging the 20-minute rule first thing in the morning, as the day is starting. However, there is a caveat to this advice as well. For example, what is the first thing an architect does after getting to work in the morning? Well, the very

first thing the architect does is to get that wonderful cup of coffee or tea. OK, in that case, what is the second thing every architect does after getting that necessary coffee or tea—check email. Once an architect checks email, diversion happens, email responses are written, and the day is over. Therefore, our strong recommendation is to invoke the 20-minute rule first thing in the morning, right after grabbing that cup of coffee or tea and before checking email. Go in to work a little early. Doing this will increase an architect's technical breadth and help develop the knowledge required to become an effective software architect.

Developing a Personal Radar

For most of the '90s and the beginning of the '00s, Neal was the CTO of a small training and consulting company. When he started there, the primary platform was Clipper, which was a rapid-application development tool for building DOS applications atop dBASE files. Until one day it vanished. The company had noticed the rise of Windows, but the business market was still DOS…until it abruptly wasn't. That lesson left a lasting impression: ignore the march of technology at your peril.

It also taught an important lesson about technology bubbles. When heavily invested in a technology, a developer lives in a memetic bubble, which also serves as an echo chamber. Bubbles created by vendors are particularly dangerous, because developers never hear honest appraisals from within the bubble. But the biggest danger of Bubble Living comes when it starts collapsing, which developers never notice from the inside until it's too late.

What they lacked was a technology radar: a living document to assess the risks and rewards of existing and nascent technologies. The radar concept comes from ThoughtWorks; first, we'll describe how this concept came to be and then how to use it to create a personal radar.

The ThoughtWorks Technology Radar

The ThoughtWorks Technology Advisory Board (TAB) is a group of senior technology leaders within ThoughtWorks, created to assist the CTO, Dr. Rebecca Parsons, in deciding technology directions and strategies for the company and its clients. This group meets face-to-face twice a year. One of the outcomes of the face to face meeting was the Technology Radar. Over time, it gradually grew into the biannual Technology Radar.

The TAB gradually settled into a twice-a-year rhythm of Radar production. Then, as often happens, unexpected side effects occurred. At some of the conferences Neal spoke at, attendees sought him out and thanked him for helping produce the Radar and said that their company had started producing their own version of it.

Neal also realized that this was the answer to a pervasive question at conference speaker panels everywhere: "How do you (the speakers) keep up with technology? How do you figure out what things to pursue next?" The answer, of course, is that they all have some form of internal radar.

Parts

The ThoughtWorks Radar consists of four quadrants that attempt to cover most of the software development landscape:

Tools
: Tools in the software development space, everything from developers tools like IDEs to enterprise-grade integration tools

Languages and frameworks
: Computer languages, libraries, and frameworks, typically open source

Techniques
: Any practice that assists software development overall; this may include software development processes, engineering practices, and advice

Platforms
: Technology platforms, including databases, cloud vendors, and operating systems

Rings

The Radar has four rings, listed here from outer to inner:

Hold
: The original intent of the hold ring was "hold off for now," to represent technologies that were too new to reasonably assess yet—technologies that were getting lots of buzz but weren't yet proven. The hold ring has evolved into indicating "don't start anything new with this technology." There's no harm in using it on existing projects, but developers should think twice about using it for new development.

Assess
: The assess ring indicates that a technology is worth exploring with the goal of understanding how it will affect an organization. Architects should invest some effort (such as development spikes, research projects, and conference sessions) to see if it will have an impact on the organization. For example, many large companies visibly went through this phase when formulating a mobile strategy.

Trial

The trial ring is for technologies worth pursuing; it is important to understand how to build up this capability. Now is the time to pilot a low-risk project so that architects and developers can really understand the technology.

Adopt

For technologies in the adopt ring, ThoughtWorks feels strongly that the industry should adopt those items.

An example view of the Radar appears in Figure 24-2.

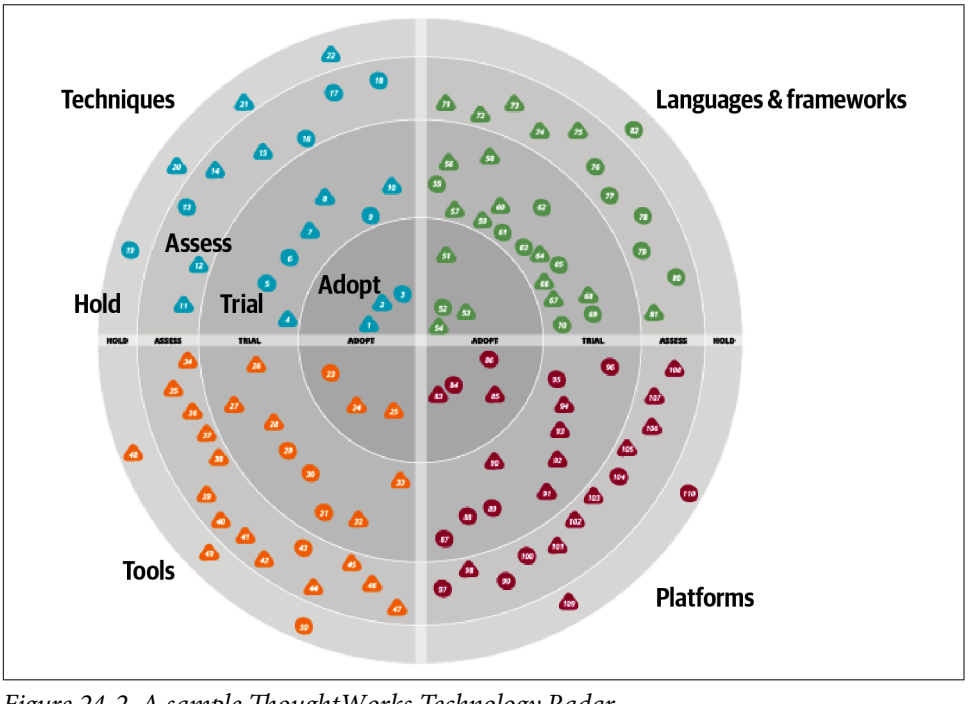

Figure 24-2. A sample ThoughtWorks Technology Radar

In Figure 24-2, each blip represents a different technology or technique, with associated short write-ups. While ThoughtWorks uses the radar to broadcast their opinions about the software world, many developers and architects also use it as a way of structuring their technology assessment process. Architects can use the tool described in "Open Source Visualization Bits" on page 371 to build the same visuals used by ThoughtWorks as a way to organize their thinking about what to invest time in.

When using the radar for personal use, we suggest altering the meanings of the quadrants to the following:

Hold

An architect can include not only technologies and techniques to avoid, but also habits they are trying to break. For example, an architect from the .NET world may be accustomed to reading the latest news/gossip on forums about team internals. While entertaining, it may be a low-value information stream. Placing that in hold forms a reminder for an architect to avoid problematic things.

Assess

Architects should use *assess* for promising technologies that they have heard good things about but haven't had time to assess for themselves yet—see "Using Social Media" on page 371. This ring forms a staging area for more serious research at some time in the future.

Trial

The *trial* ring indicates active research and development, such as an architect performing spike experiments within a larger code base. This ring represents technologies worth spending time on to understand more deeply so that an architect can perform an effective trade-off analysis.

Adopt

The *adopt* ring represents the new things an architect is most excited about and best practices for solving particular problems.

It is dangerous to adopt a laissez-faire attitude toward a technology portfolio. Most technologists pick technologies on a more or less ad hoc basis, based on what's cool or what your employer is driving. Creating a technology radar helps an architect formalize their thinking about technology and balance opposing decision criteria (such as the "more cool" technology factor and being less likely to get a new job versus a huge job market but with less interesting work). Architects should treat their technology portfolio like a financial portfolio: in many ways, they are the same thing. What does a financial planner tell people about their portfolio? Diversify!

Architects should choose some technologies and/or skills that are widely in demand and track that demand. But they might also want to try some technology gambits, like open source or mobile development. Anecdotes abound about developers who freed themselves from cubicle-dwelling servitude by working late at night on open source projects that became popular, purchasable, and eventually, career destinations. This is yet another reason to focus on breadth rather than depth.

Architects should set aside time to broaden their technology portfolio, and building a radar provides a good scaffolding. However, the exercise is more important than the outcome. Creating the visualization provides an excuse to think about these things,

and, for busy architects, finding an excuse to carve out time in a busy schedule is the only way this kind of thinking can occur.

Open Source Visualization Bits

By popular demand, ThoughtWorks released a tool in November 2016 to assist technologists in building their own radar visualization. When ThoughtWorks does this exercise for companies, they capture the output of the meeting in a spreadsheet, with a page for each quadrant. The ThoughtWorks Build Your Own Radar tool uses a Google spreadsheet as input and generates the radar visualization using an HTML 5 canvas. Thus, while the important part of the exercise is the conversations it generates, it also generates useful visualizations.

Using Social Media

Where can an architect find new technologies and techniques to put in the assess ring of their radar? In Andrew McAfee's book *Enterprise 2.0* (Harvard Business Review Press), he makes an interesting observation about social media and social networks in general. When thinking about a person's network of contact between people, three categories exist, as illustrated in Figure 24-3.

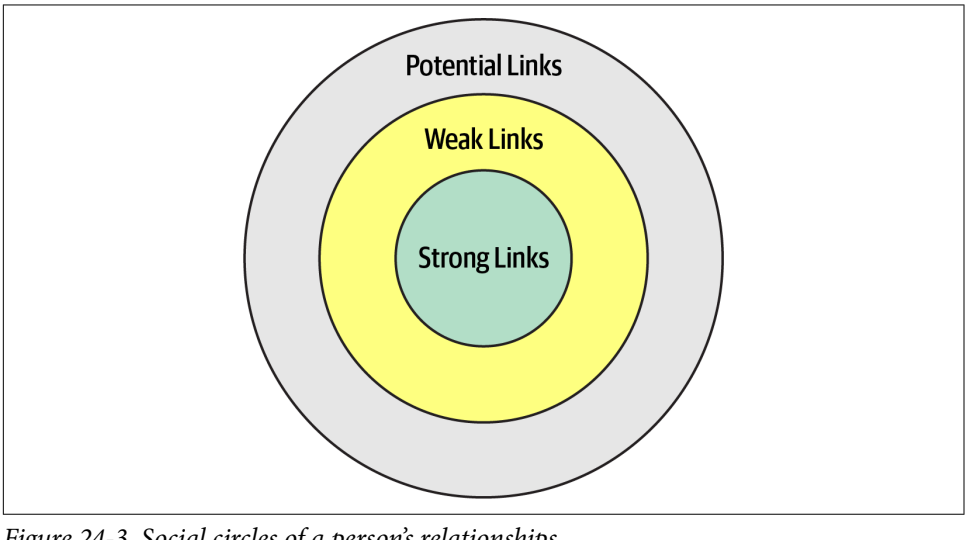

Figure 24-3. Social circles of a person's relationships

In Figure 24-3, *strong links* represent family members, coworkers, and other people whom a person regularly contacts. One litmus test for how close these connections are: they can tell you what a person in their strong links had for lunch at least one day last week. *Weak links* are casual acquaintances, distant relatives, and other people seen

only a few times a year. Before social media, it was difficult to keep up with this circle of people. Finally, *potential links* represent people you haven't met yet.

McAfee's most interesting observation about these connections was that someone's next job is more likely to come from a weak link than a strong one. Strongly linked people know everything within the strongly linked group—these are people who see each other all the time. Weak links, on the other hand, offer advice from outside someone's normal experience, including new job offers.

Using the characteristics of social networks, architects can utilize social media to enhance their technical breadth. Using social media like Twitter professionally, architects should find technologists whose advice they respect and follow them on social media. This allows an architect to build a network on new, interesting technologies to assess and keep up with the rapid changes in the technology world.

Parting Words of Advice

> How do we get great designers? Great designers design, of course.
>
> —Fred Brooks

> So how are we supposed to get great architects, if they only get the chance to architect fewer than a half-dozen times in their career?
>
> —Ted Neward

Practice is the proven way to build skills and become better at anything in life… including architecture. We encourage new and existing architects to keep honing their skills, both for individual technology breadth but also for the craft of designing architecture. To that end, check out the architecture katas on the companion website for the book. Modeled after the katas used as examples here, we encourage architects to use these to practice building skills in architecture.

A common question we get about katas: is there an answer guide somewhere? Unfortunately such an answer key does not exist. To quote your author, Neal:

> There are not right or wrong answers in architecture—only *trade-offs*.

When we started using the architecture katas exercise during live training classes, we initially kept the drawings the students produced with the goal of creating an answer repository. We quickly gave up, though, because we realized that we had incomplete artifacts. In other words, the teams had captured the topology and explained their decisions in class but didn't have the time to create architecture decision records. While how they implemented their solutions was interesting, the why was much more interesting because it contains the trade-offs they considered in making that decision. Keeping just the how was only half of the story. So, our last parting words of advice: always learn, always practice, and *go do some architecture*!

APPENDIX

Self-Assessment Questions

Chapter 1: Introduction

1. What are the four dimensions that define software architecture?
2. What is the difference between an architecture decision and a design principle?
3. List the eight core expectations of a software architect.
4. What is the First Law of Software Architecture?

Chapter 2: Architectural Thinking

1. Describe the traditional approach of architecture versus development and explain why that approach no longer works.
2. List the three levels of knowledge in the knowledge triangle and provide an example of each.
3. Why is it more important for an architect to focus on technical breadth rather than technical depth?
4. What are some of the ways of maintaining your technical depth and remaining hands-on as an architect?

Chapter 3: Modularity

1. What is meant by the term *connascence*?
2. What is the difference between static and dynamic connascence?
3. What does *connascence of type* mean? Is it static or dynamic connascence?
4. What is the strongest form of connascence?
5. What is the weakest form of connascence?
6. Which is preferred within a code base—static or dynamic connascence?

Chapter 4: Architecture Characteristics Defined

1. What three criteria must an attribute meet to be considered an architecture characteristic?
2. What is the difference between an implicit characteristic and an explicit one? Provide an example of each.
3. Provide an example of an operational characteristic.
4. Provide an example of a structural characteristic.
5. Provide an example of a cross-cutting characteristic.
6. Which architecture characteristic is more important to strive for—availability or performance?

Chapter 5: Identifying Architecture Characteristics

1. Give a reason why it is a good practice to limit the number of characteristics ("-ilities") an architecture should support.
2. True or false: most architecture characteristics come from business requirements and user stories.
3. If a business stakeholder states that time-to-market (i.e., getting new features and bug fixes pushed out to users as fast as possible) is the most important business concern, which architecture characteristics would the architecture need to support?
4. What is the difference between scalability and elasticity?
5. You find out that your company is about to undergo several major acquisitions to significantly increase its customer base. Which architectural characteristics should you be worried about?

Chapter 6: Measuring and Governing Architecture Characteristics

1. Why is cyclomatic complexity such an important metric to analyze for architecture?
2. What is an architecture fitness function? How can they be used to analyze an architecture?
3. Provide an example of an architecture fitness function to measure the scalability of an architecture.
4. What is the most important criteria for an architecture characteristic to allow architects and developers to create fitness functions?

Chapter 7: Scope of Architecture Characteristics

1. What is an architectural quantum, and why is it important to architecture?
2. Assume a system consisting of a single user interface with four independently deployed services, each containing its own separate database. Would this system have a single quantum or four quanta? Why?
3. Assume a system with an administration portion managing static reference data (such as the product catalog, and warehouse information) and a customer-facing portion managing the placement of orders. How many quanta should this system be and why? If you envision multiple quanta, could the admin quantum and customer-facing quantum share a database? If so, in which quantum would the database need to reside?

Chapter 8: Component-Based Thinking

1. We define the term *component* as a building block of an application—something the application does. A component usually consist of a group of classes or source files. How are components typically manifested within an application or service?
2. What is the difference between technical partitioning and domain partitioning? Provide an example of each.
3. What is the advantage of domain partitioning?
4. Under what circumstances would technical partitioning be a better choice over domain partitioning?
5. What is the entity trap? Why is it not a good approach for component identification?

6. When might you choose the workflow approach over the Actor/Actions approach when identifying core components?

Chapter 9: Architecture Styles

1. List the eight fallacies of distributed computing.
2. Name three challenges that distributed architectures have that monolithic architectures don't.
3. What is stamp coupling?
4. What are some ways of addressing stamp coupling?

Chapter 10: Layered Architecture Style

1. What is the difference between an open layer and a closed layer?
2. Describe the layers of isolation concept and what the benefits are of this concept.
3. What is the architecture sinkhole anti-pattern?
4. What are some of the main architecture characteristics that would drive you to use a layered architecture?
5. Why isn't testability well supported in the layered architecture style?
6. Why isn't agility well supported in the layered architecture style?

Chapter 11: Pipeline Architecture

1. Can pipes be bidirectional in a pipeline architecture?
2. Name the four types of filters and their purpose.
3. Can a filter send data out through multiple pipes?
4. Is the pipeline architecture style technically partitioned or domain partitioned?
5. In what way does the pipeline architecture support modularity?
6. Provide two examples of the pipeline architecture style.

Chapter 12: Microkernel Architecture

1. What is another name for the microkernel architecture style?
2. Under what situations is it OK for plug-in components to be dependent on other plug-in components?
3. What are some of the tools and frameworks that can be used to manage plug-ins?
4. What would you do if you had a third-party plug-in that didn't conform to the standard plug-in contract in the core system?
5. Provide two examples of the microkernel architecture style.
6. Is the microkernel architecture style technically partitioned or domain partitioned?
7. Why is the microkernel architecture always a single architecture quantum?
8. What is domain/architecture isomorphism?

Chapter 13: Service-Based Architecture

1. How many services are there in a typical service-based architecture?
2. Do you have to break apart a database in service-based architecture?
3. Under what circumstances might you want to break apart a database?
4. What technique can you use to manage database changes within a service-based architecture?
5. Do domain services require a container (such as Docker) to run?
6. Which architecture characteristics are well supported by the service-based architecture style?
7. Why isn't elasticity well supported in a service-based architecture?
8. How can you increase the number of architecture quanta in a service-based architecture?

Chapter 14: Event-Driven Architecture Style

1. What are the primary differences between the broker and mediator topologies?
2. For better workflow control, would you use the mediator or broker topology?
3. Does the broker topology usually leverage a publish-and-subscribe model with topics or a point-to-point model with queues?
4. Name two primary advantage of asynchronous communications.
5. Give an example of a typical request within the request-based model.

6. Give an example of a typical request in an event-based model.
7. What is the difference between an initiating event and a processing event in event-driven architecture?
8. What are some of the techniques for preventing data loss when sending and receiving messages from a queue?
9. What are three main driving architecture characteristics for using event-driven architecture?
10. What are some of the architecture characteristics that are not well supported in event-driven architecture?

Chapter 15: Space-Based Architecture

1. Where does space-based architecture get its name from?
2. What is a primary aspect of space-based architecture that differentiates it from other architecture styles?
3. Name the four components that make up the virtualized middleware within a space-based architecture.
4. What is the role of the messaging grid?
5. What is the role of a data writer in space-based architecture?
6. Under what conditions would a service need to access data through the data reader?
7. Does a small cache size increase or decrease the chances for a data collision?
8. What is the difference between a replicated cache and a distributed cache? Which one is typically used in space-based architecture?
9. List three of the most strongly supported architecture characteristics in space-based architecture.
10. Why does testability rate so low for space-based architecture?

Chapter 16: Orchestration-Driven Service-Oriented Architecture

1. What was the main driving force behind service-oriented architecture?
2. What are the four primary service types within a service-oriented architecture?
3. List some of the factors that led to the downfall of service-oriented architecture.
4. Is service-oriented architecture technically partitioned or domain partitioned?
5. How is domain reuse addressed in SOA? How is operational reuse addressed?

Chapter 17: Microservices Architecture

1. Why is the bounded context concept so critical for microservices architecture?
2. What are three ways of determining if you have the right level of granularity in a microservice?
3. What functionality might be contained within a sidecar?
4. What is the difference between orchestration and choreography? Which does microservices support? Is one communication style easier in microservices?
5. What is a saga in microservices?
6. Why are agility, testability, and deployability so well supported in microservices?
7. What are two reasons performance is usually an issue in microservices?
8. Is microservices a domain-partitioned architecture or a technically partitioned one?
9. Describe a topology where a microservices ecosystem might be only a single quantum.
10. How was domain reuse addressed in microservices? How was operational reuse addressed?

Chapter 18: Choosing the Appropriate Architecture Style

1. In what way does the data architecture (structure of the logical and physical data models) influence the choice of architecture style?
2. How does it influence your choice of architecture style to use?
3. Delineate the steps an architect uses to determine style of architecture, data partitioning, and communication styles.
4. What factor leads an architect toward a distributed architecture?

Chapter 19: Architecture Decisions

1. What is the covering your assets anti-pattern?
2. What are some techniques for avoiding the email-driven architecture anti-pattern?
3. What are the five factors Michael Nygard defines for identifying something as architecturally significant?
4. What are the five basic sections of an architecture decision record?
5. In which section of an ADR do you typically add the justification for an architecture decision?
6. Assuming you don't need a separate Alternatives section, in which section of an ADR would you list the alternatives to your proposed solution?
7. What are three basic criteria in which you would mark the status of an ADR as Proposed?

Chapter 20: Analyzing Architecture Risk

1. What are the two dimensions of the risk assessment matrix?
2. What are some ways to show direction of particular risk within a risk assessment? Can you think of other ways to indicate whether risk is getting better or worse?
3. Why is it necessary for risk storming to be a collaborative exercise?
4. Why is it necessary for the identification activity within risk storming to be an individual activity and not a collaborative one?
5. What would you do if three participants identified risk as high (6) for a particular area of the architecture, but another participant identified it as only medium (3)?
6. What risk rating (1-9) would you assign to unproven or unknown technologies?

Chapter 21: Diagramming and Presenting Architecture

1. What is irrational artifact attachment, and why is it significant with respect to documenting and diagramming architecture?
2. What do the 4 C's refer to in the C4 modeling technique?
3. When diagramming architecture, what do dotted lines between components mean?
4. What is the bullet-riddled corpse anti-pattern? How can you avoid this anti-pattern when creating presentations?

5. What are the two primary information channels a presenter has when giving a presentation?

Chapter 22: Making Teams Effective

1. What are three types of architecture personalities? What type of boundary does each personality create?
2. What are the five factors that go into determining the level of control you should exhibit on the team?
3. What are three warning signs you can look at to determine if your team is getting too big?
4. List three basic checklists that would be good for a development team.

Chapter 23: Negotiation and Leadership Skills

1. Why is negotiation so important as an architect?
2. Name some negotiation techniques when a business stakeholder insists on five nines of availability, but only three nines are really needed.
3. What can you derive from a business stakeholder telling you "I needed it yesterday"?
4. Why is it important to save a discussion about time and cost for last in a negotiation?
5. What is the divide-and-conquer rule? How can it be applied when negotiating architecture characteristics with a business stakeholder? Provide an example.
6. List the 4 C's of architecture.
7. Explain why it is important for an architect to be both pragmatic and visionary.
8. What are some techniques for managing and reducing the number of meetings you are invited to?

Chapter 24: Developing a Career Path

1. What is the 20-minute rule, and when is it best to apply it?
2. What are the four rings in the ThoughtWorks technology radar, and what do they mean? How can they be applied to your radar?
3. Describe the difference between depth and breadth of knowledge as it applies to software architects. Which should architects aspire to maximize?